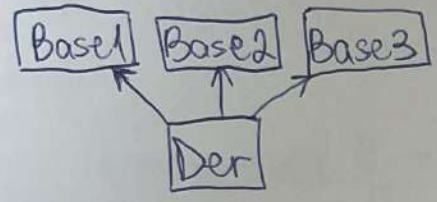
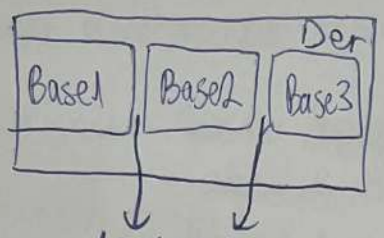
Тема 12

• множествено наследяване



Der d;



може да има padding

```
Base1* ptr1 = &d;
Base2* ptr2 = &d;
Base3* ptr3 = &d;
```

} използват специален механизъм, който знае колко да се отместят

↳ и трите са <u>валидни</u>

&d = ptr1; ptr1 ≠ ptr2; ptr2 ≠ ptr3; ptr3 ≠ ptr1

• Der() е отговорен за Base1, Base2 и Base3

```
Der:Base1, Base2, Base3{ A a; B b; }
```

→ Base1() Base2() Base3() A() B() Der() - не зависи от инициал. списъка

деструк. са в обратен ред

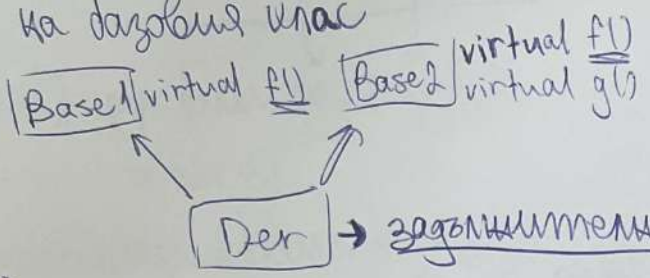
```
OP= {
  if(this != &other){
      Base1::OP=(other);
      Base2::OP=(other);
      Base3::OP=(other);
      free();
      copyFrom(other);
  }
}
```

адресът на other се мести заради Base2 и Base3

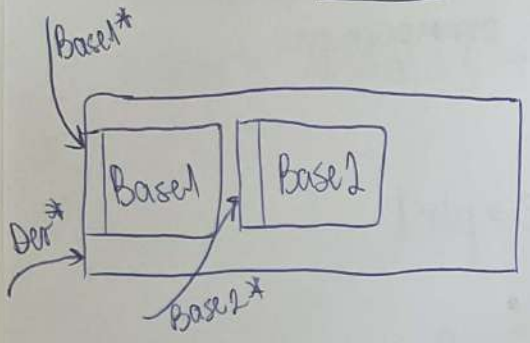
• виртуални таблици

→ функция Δ – Δ(BaseN) – показва отместването на BaseN от началото на Der

→ всеки базов клас има vtable на Der в контекста на базовия клас



→ задължително override-ва f()



Base1 vtable:

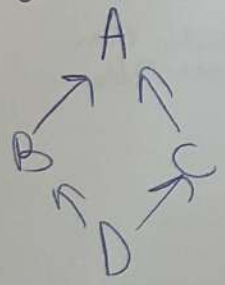
Der::f()	-Δ(Base1) = 0

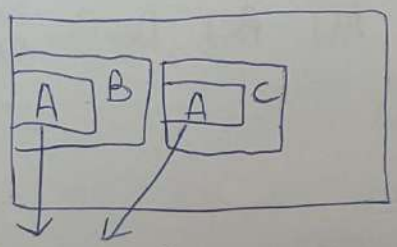
Base 2 vtable:

Base2::g()	0
Der::f()	-Δ(Base2)

с тяхова трябва да се отчетат указателят

• диамантен проблем



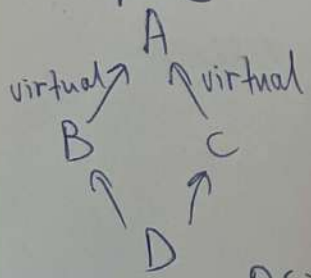


в D има 2x A

D(): B(), C(), ~~A()~~ {..}

D не може да каже как да бъде създадено A

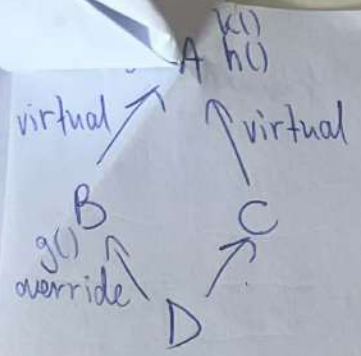
• виртуално наследяване – class X: virtual Y {..};

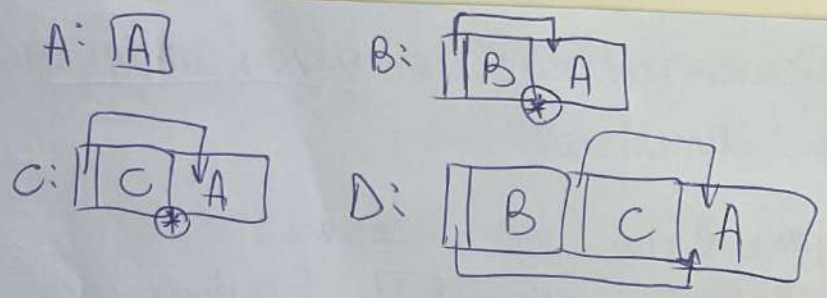


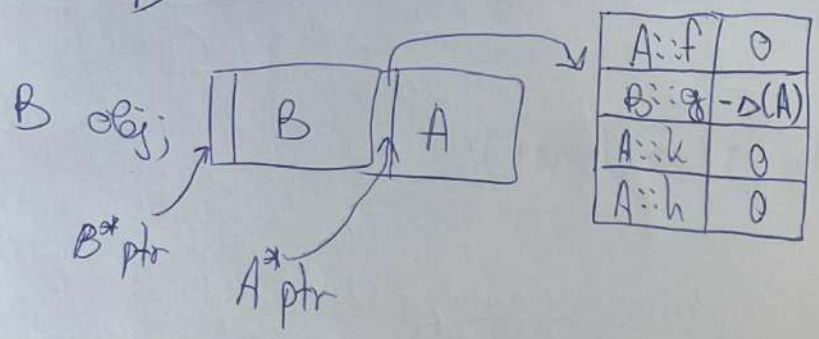
Всеки прек и непрек наследник на X е длъжен да каже как да бъде създаден Y (вика се default Y(), ако не е посочено; не се комп., ако няма default Y())

D(): B(), C(), A() {...}

D е отговорен за създаването на A

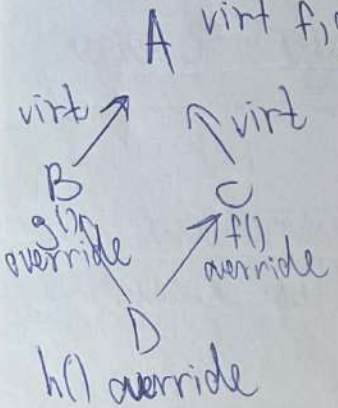


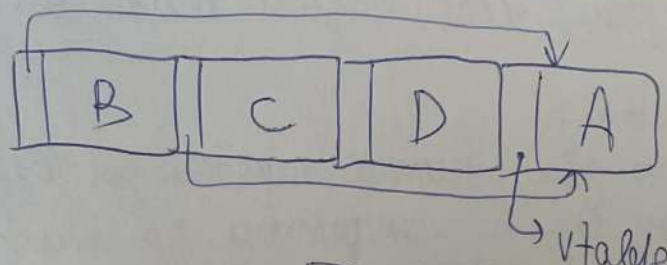




A::f	0
B::g	-Δ(A)
A::k	0
A::h	0

(*) A може да не е "залепен" до наследника си, а да има място между тях



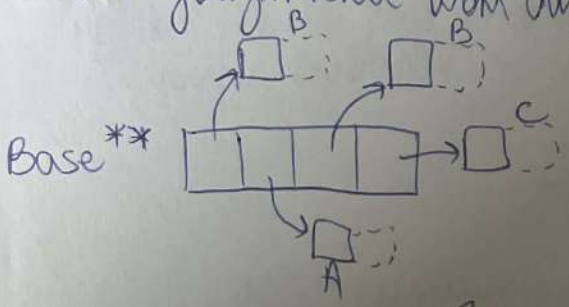


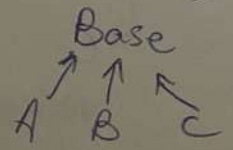
vtable D:

C::f	-Δ(A)+Δ(C)
B::g	-Δ(A)
A::k	0
D::h	-Δ(A)

• колекция от обекти в полиморфна йерархия

→ хетерогенен контейнер-клас, който съдържа колекция от указатели към базов абстрактен клас





→ триене - чрез виртуален деструктор

delete data[i] → вика virtual ~Base() → вика конкретен деструктор → в края си той прочете виртуалния указ. → извиква се ~Base()

• копиране – чисто-виртуална клонираща функция

Base:
```
Base* clone() const = 0;
```
↓

Der1: (и всеки друг наследник)
```
Base* clone() const override { return new Der1(*this); }
```

копиране в XK:
```
data[i] = other.data[i] -> clone();
```

• разпознаване
→ dynamic_cast
→ в базовия клас пазим типа на текущия обект

• visitor pattern
→ когато два обекта трябва да взаимодействат помежду си с конкретна за типовете им логика

Base:
```
virtual handleOther(Base* other)
virtual handleA(A* other)
virtual handleB(B* other)
virtual handleC(C* other)
```
A ↑ B ↑ C

A:
```
handleOther(Base* other)
    other -> handleA(this);
```
→ разбрах сме, че this е от тип A
→ сега викаме handleA, чрез other, за да разберем и неговия тип

C:
```
handleA(A* other)
//конкретна логика за A и C
```

→ класът трябва да знае за наследниците си
⇒ forward декларация

Base.h
```
class A;
class B;
class C;
```
→ могат да се ползват само указатели и референции на forward декларираните класове